

Automāti

Laika ierobežojums: 2 sekundes

Atmiņas ierobežojums: 1024 MiB

Ir dots lauks, kas sastāv no N šūnām, kas izvietotas rindā un numurētas no 0 līdz $N - 1$ virzienā no kreisās puses uz labo. Katrai šūnai ir atšķirīgs augstums: šūnas i augstums ir p_i , un virkne $p_0, p_1, \dots, p_{N-1}$ ir skaitļu $0, 1, \dots, N - 1$ permutācija.

Divas šūnas ar **indeksiem** i un j sauc par *tuvām* tad un tikai tad, ja $|i - j| \leq 1$. Katra šūna ir tuva pati sev.

Uz lauka atrodas robots, kurš sākotnēji novietots kādā šūnā. Robots pieņem šādu divu veidu komandas:

- **MAX**: starp visām šūnām, kas ir tuvas robota pašreizējai šūnai, robota nākamā pozīcija būs vienīgajā šūnā ar **maksimālo** augstumu;
- **MIN**: starp visām šūnām, kas ir tuvas robota pašreizējai šūnai, robota nākamā pozīcija būs vienīgajā šūnā ar **minimālo** augstumu.

Tā kā šūna ir tuva pati sev, komandas rezultātā robots var palikt tajā pašā šūnā.

Programma ir galīga komandu virkne, kurā katra komanda ir vai nu **MAX**, vai **MIN**. Ja robots sāk šūnā X un izpilda programmu S , tas beigās nonāk kādā viennozīmīgi noteiktā šūnā, ko apzīmē ar $\text{result}(S, X)$.

Jums tiks uzdoti Q vaicājumi, katrā no tiem dota kāda sākuma šūnu kopa ar izmēru K_i ($0 \leq i < Q$). i -tais vaicājums būs par kādām šūnām $x_0, x_1, \dots, x_{K_i-1}$; robots sākumā tiks novietots vienā no tām, taču iepriekš nav zināms, kurā tieši. Katram vaicājumam jums jānosaka, vai eksistē programma, kas pārvieto robotu uz vienu un to pašu beigu šūnu neatkarīgi no tā, kurā šūnā no x robots sāka izpildi.

Formāli runājot, jums jānosaka, vai eksistē tāda programma S , ka:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Permutācija p ir **viena un tā pati** visiem vaicājumiem.

Ievērojiet, ka šāda programma nav jākonstruē – jāatgriež tikai atbilde, vai tāda eksistē.

Implementācijas detaļas

Pirmā funkcija, kas jums jāimplementē, ir:

```
void initialize(std::vector<int> p)
```

- p : skaitļu no 0 līdz $N - 1$ permutācija.

Otrā funkcija, kas jums jāimplementē, ir:

```
bool exists_program(std::vector<int> x)
```

- x : masīvs, kas stingri augošā secībā norāda vaicājumā dotās šūnas.

Funkcija `initialize` tiek izsaukta tieši vienu reizi, pirms tiek veikts jebkurš funkcijas `exists_program` izsaukums.

Funkcija `exists_program` tiek izsaukta Q reizes, pa vienai reizei katram vaicājumam. Tai jāatgriež `true`, ja eksistē programma, pēc kuras robots nonāk vienā un tajā pašā šūnā neatkarīgi no tā, kurā no dotajām šūnām tas sāka, un `false` pretējā gadījumā.

Ierobežojumi

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ ir skaitļu $0, 1, \dots, N - 1$ permutācija
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ katram vaicājumam
- K_i summa pa visiem Q vaicājumiem nepārsniedz 10^6 .

Piemēri

Parauga vērtētāja ievaddati 1	Parauga vērtētāja izvaddati 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Parauga vērtētāja ievaddati 2	Parauga vērtētāja izvaddati 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

1. piemēra izskaidrojums

Šajā piemērā $p = [0, 2, 1]$. Otrajā vaicājumā robots var sākt šūnā 0, šūnā 1 vai šūnā 2. Aplūkosim programmu `[MAX]`.

- Ja robots sāk šūnā 0: šūnai 0 tuvās šūnas ir šūnas ar indeksiem 0 un 1. Izpildot komandu `MAX`, tā kā $p_0 < p_1$, robots pārvietojas uz šūnu 1.
- Ja robots sāk šūnā 1: šūnai 1 tuvās šūnas ir šūnas ar indeksiem 0, 1 un 2. Izpildot komandu `MAX`, tā kā $p_0 < p_1$ un $p_1 > p_2$, robots paliek šūnā 1.
- Ja robots sāk šūnā 2: šūnai 2 tuvās šūnas ir šūnas ar indeksiem 1 un 2. Izpildot komandu `MAX`, tā kā $p_1 > p_2$, robots pārvietojas uz šūnu 1.

Tātad eksistē programma, pēc kuras izpildes robots no visām trim sākuma šūnām nonāk šūnā 1, un atbilde uz šo vaicājumu ir `true`. Der arī citas programmas, piemēram, `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` un `[MAX, MIN, MAX]`.

2. piemēra skaidrojums

Šajā piemērā $p = [0, 4, 2, 1, 3, 5, 6]$.

Par pirmajiem diviem vaicājumiem var pierādīt, ka neeksistē programma, pēc kuras robots no katras dotās sākuma šūnas nonāktu vienā un tajā pašā šūnā, tāpēc atbilde uz abiem ir `false`.

Aplūkosim pēdējo vaicājumu, kurā robots var sākt šūnā 3 vai šūnā 6, un aplūkosim programmu `[MAX, MAX, MAX]`.

- Ja robots sāk šūnā 3: šūnai 3 tuvās šūnas ir šūnas ar indeksiem 2, 3 un 4. Tā kā $p_3 < p_4$ un $p_2 < p_4$, robots pārvietojas uz šūnu 4. Tādā pašā veidā izpildot nākamās divas komandas, tas nonāk šūnā 6.
- Ja robots sāk šūnā 6, tas visu laiku paliek šūnā 6 un tur arī nonāk beigās.

Tātad atbilde uz šo vaicājumu ir `true`. Der arī programma `[MIN, MAX, MAX, MAX]`. Tomēr citas programmas var arī nederēt. Piemēram, ja programma ir `[MAX, MAX]`, tad, sākot šūnā 3, robots nonāk šūnā 5, bet, sākot šūnā 6, tas nonāk šūnā 6.

Apakšuzdevumi

Apakšuzd.	Punkti	N	Q	K_i	Papildu ierobežojumi
0	0	–	–	–	Piemēri.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Ja atbilde uz vaicājumu ir apstiprinoša, tad eksistē derīga programma, kas izmanto tieši 1 komandu.
2	7				Ja atbilde uz vaicājumu ir apstiprinoša, tad eksistē derīga programma, kas izmanto ne vairāk kā 5 komandas.
3	9	≤ 100	≤ 500		–
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$		–
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ katram vaicājumam.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$		Permutācija sākumā ir augoša, tad dilstoša, tad atkal augoša. Formāli runājot, eksistē tādi veseli skaitļi a, b , ka $0 < a < b < N - 1$ un $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$, ja N ir pāra skaitlis, vai $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$, ja N ir nepāra skaitlis.
9	23				–

Piemēra vērtētājprogramma

Ievaddatu formāts ir šāds:

- 1. rinda: divi veseli skaitļi – N un Q vērtības;
- 2. rinda: N veseli skaitļi $p_0, p_1, \dots, p_{N-1}$, kur p_i ir i -tās šūnas augstums;
- 3 + i . rinda: i -tajā vaicājumā dotās šūnas – vesels skaitlis K_i , kam tajā pašā rindā seko K_i veseli skaitļi $x_0, x_1, \dots, x_{K_i-1}$ – robota iespējamās sākuma šūnas.

Izvaddatu formāts ir šāds:

- 1. rinda: binārā virkne ar garumu Q , kuras i -tais simbols ir 1, ja atbilde uz i -to vaicājumu ir `true`, un 0 pretējā gadījumā.